

SYSTEM ARCHITECTURE AND SOFTWARE DESIGN

Engineering & Architecture Team

MICROFI

System Architecture and Software Design Document
Modular Monolith Edition

ARCHITECTURE	VERSION
Modular Monolith + Event-Driven Hybrid	1.1

Table of Contents

0. Contents

1	System Design Methodology	3
1.1	Inputs	3
1.2	Design Principles	3
1.3	Method Steps	3
2	Global System Architecture	4
3	Technology Stack (Detailed by Component)	5
3.1	Mobile Application (Agent + Client)	5
3.2	Back Office (Web)	5
3.3	API Gateway	6
3.4	Core Backend (Business Logic)	6
3.5	CBS Middleware (Integration Service)	7
3.6	Data & Messaging Infrastructure	8
3.7	Containerisation & Ops	8
3.8	Observability Stack	9
4	Component Breakdown Summary	10
5	Internal Modular Structure (Core Backend)	10
5.1	Directory Layout	10
5.2	Middleware Service Layout	11
5.3	Core Module Responsibilities	11
6	Database Schema	12
6.1	Entity Relationship Overview	12
6.2	Schema core — Identity & Organisation	13
6.3	Schema core — Escrow (FR-03, FR-04, FR-05)	13
6.4	Schema core — Client Digital Booklet (FR-19–FR-22)	13
6.5	Schema core — Collections & Tracking (FR-06–FR-14)	14
6.6	Schema core — End-of-Day OFJ (FR-16–FR-18)	14
6.7	Schema mw — Middleware Idempotency & Mapping	15
6.8	Illustrative DDL (excerpts)	15
7	STRIDE Security Matrix	17
7.1	Summary Matrix	17
7.2	S — Spoofing (Authenticity)	17
7.3	T — Tampering (Integrity)	18
7.4	R — Repudiation (Non-repudiability)	18
7.5	I — Information Disclosure (Confidentiality)	18
7.6	D — Denial of Service (Availability)	19
7.7	E — Elevation of Privilege (Authorization)	19
7.8	Traceability to Design Artifacts	19

- 8 Use Case Descriptions (from Business Analysis) 19**
 - 8.1 Agent Access & Administration 20
 - 8.2 Escrow & Ceilings 20
 - 8.3 Collection Operations 21
 - 8.4 Field Tracking & Security 22
 - 8.5 End-of-Day (OFJ) 23
 - 8.6 Client Digital Booklet & Self-Service 24
- 9 Human-Machine Interface (HMI) Design Choices 25**
- 10 Production-Ready Docker Compose Configuration 26**
- 11 Key API Endpoints 29**
 - 11.1 Core Backend (API Gateway surface) 29
 - 11.2 CBS Middleware (Spring Boot 4 + JPA) — internal 30

1. System Design Methodology

This document is the **System Architecture and Software Design Document** (SAD) for MICROFI. It translates the Business Analysis & Requirements Specification (v3.1) into a concrete technical architecture, technology stack, data model, interface contracts and deployable topology.

Inputs

- Business Analysis & Requirements Specification (AS-IS / TO-BE, stakeholders, RACI, FR-01 to FR-22, NFR-01 to NFR-12, BR-01 to BR-04).
- Feature Priority Matrix and MVP boundary (6-week production window).
- CEMAC operational constraints: intermittent connectivity, low-literacy field users, COBAC/BEAC traceability, agent commission economics (digital booklet).

Design Principles

1. **Requirements traceability** — Every architectural choice maps back to one or more FRs, NFRs or BRs (e.g. offline-first ↔ FR-07 / NFR-01 / NFR-08; tokenised booklet ↔ FR-19 / BR-04).
2. **Modular monolith + isolated CBS adapter** — Core business logic stays in one Spring Boot deployable with strict internal modules; vendor-specific CBS protocols live in a separate Spring Boot middleware so the core never depends on Amplitude/FinanSoft wire formats.
3. **Hybrid reactive / ACID** — High-concurrency paths (sync, auth, notifications) use WebFlux; money-moving paths use Spring Data JPA for strict transactional integrity.
4. **Offline-first at the edge** — The Flutter agent app is the system of record during the day; the server reconciles on sync and at OFJ.
5. **Security & compliance by design** — STRIDE-driven controls: IMEI-bound sessions, JWT authenticity, AES-256 offline store, extbfmandatory GPS gate before any collection (BR-05 / FR-12), non-repudiable geotags and OFJ audit, TLS everywhere, branch-scoped authorization, idempotent CBS posts; no mobile creation of CBS customers.
6. **Observable operations** — Metrics (Prometheus), logs (Loki), traces (Tempo) unified in Grafana from day one.

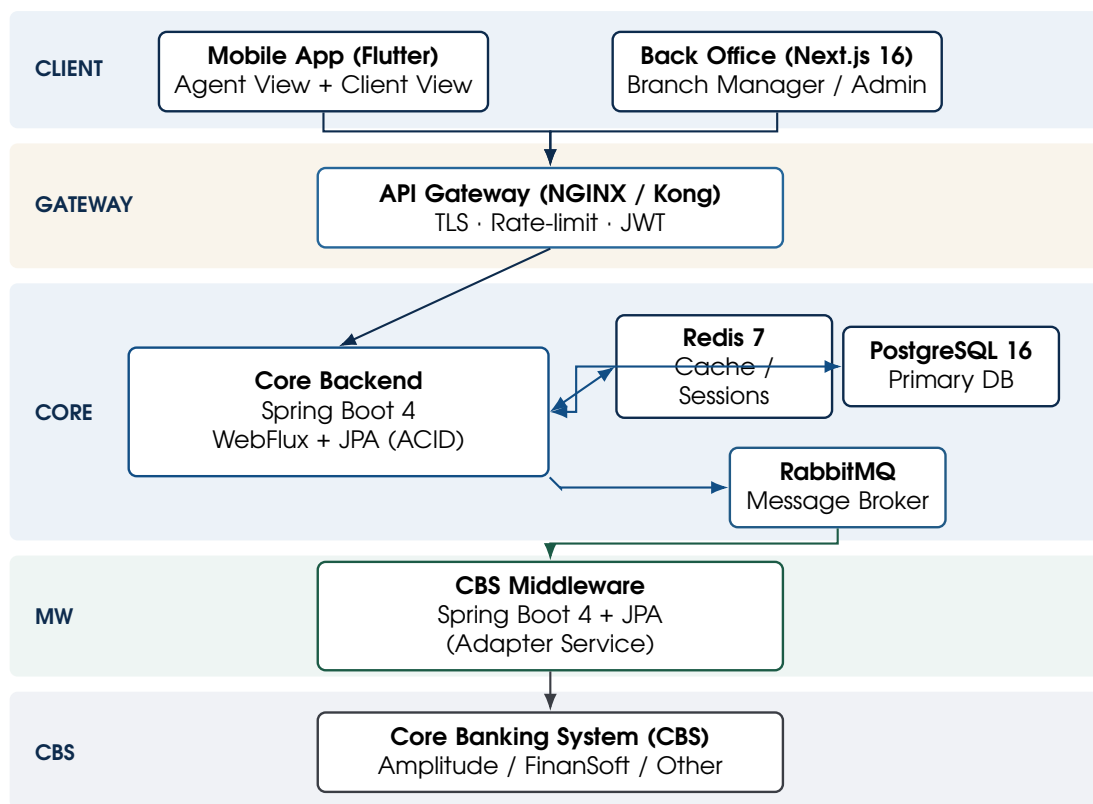
Method Steps

1. Derive logical components from the TO-BE process and FR modules (agent terminal, client app, back-office, core API, middleware, CBS).
2. Allocate each FR to a component and to an API surface (Core /api/v1 vs Middleware /mw/v1).
3. Define the relational schema that materialises entities implied by use cases (agent, collection, denomination, ofj, access_token, ...).
4. Specify the technology stack per component and the production topology (Docker Compose + observability).
5. Validate capacity and MVP scope against the Feature Priority Matrix (agent Digital Cash Desk loop first; client activation and history as Fast-Follow where scored).

This SAD does not replace the Business Analysis document. Functional intent, business rules and acceptance criteria remain authoritative in the BA; this document specifies *how* those requirements are realised in software.

2. Global System Architecture

The platform follows a **Modular Monolith & Event-Driven Hybrid Architecture**. Both the core business backend and the CBS integration middleware are implemented in **Java Spring Boot 4** with Spring Data JPA. This keeps the entire server-side stack homogeneous, simplifies deployment, and guarantees ACID consistency end-to-end while isolating CBS-specific adapters in a dedicated service.



Design principle: The **Core Backend** owns business rules, offline sync, escrow, OFJ and client tokens. The **CBS Middleware** (separate Spring Boot 4 + JPA service) owns all vendor-specific mapping and outbound calls to Amplitude, FinanSoft or other CBS products. RabbitMQ decouples the two so mobile requests are never blocked on the CBS.

3. Technology Stack (Detailed by Component)

Every tier is specified below with runtime, key libraries and an explicit **rationale** aligned to CEMAC field constraints (offline-first, low digital literacy, regulatory traceability, modest ops capacity for a pilot).

Mobile Application (Agent + Client)

Item	Choice
Framework	Flutter 3.x (Dart) — single codebase for Agent and Client views
Local storage	SQLCipher / Hive (AES-encrypted offline queue)
Auth	Biometric + PIN; IMEI binding for agents
Printing	Native Bluetooth thermal printer plugins
Maps / GPS	Geolocator + platform fused location APIs; collection UI blocked if GPS off (BR-05)
State management	Riverpod or Bloc

Why these choices (advantages).

- **Flutter** produces near-native performance on **Android and iOS** from a *single* codebase for agent and client roles. The CEMAC field pilot targets **Android first** (dominant handset class for agents), while the same project can ship an iOS build later for supervisors or clients without rewriting the UI layer. Shared plugins cover GPS, biometrics and Bluetooth printing on both platforms, cutting delivery time versus two fully separate native applications (Kotlin/Swift).
- **SQLCipher / encrypted Hive** give bank-grade **AES-256 at rest** on the handset. Offline collections and tracking points remain unreadable if the device is lost or seized — a clear advantage over plain SQLite or unencrypted local files.
- **IMEI binding + biometric/PIN** ties the session to a physical terminal and a human factor, which is stronger against shared-password or cloned-app abuse than soft login alone.
- **Fused location with a hard UI gate (BR-05)** enforces geotagging without continuous GPS streaming, preserving the ≥ 8 h autonomy target (NFR-09) that always-on tracking would jeopardise.

Back Office (Web)

Item	Choice
Framework	Next.js 16 (React 19)
Styling	Tailwind CSS
Charts / maps	Recharts + Mapbox / Leaflet
Auth client	JWT stored in httpOnly cookies

Why these choices (advantages).

- **Next.js (React)** is optimised for browser-based operational UIs (OFJ, ceilings, maps) with server rendering and a large component ecosystem — faster to ship for office users than wrapping the same screens inside a mobile shell.
- **httpOnly cookies** for JWT keep tokens inaccessible to JavaScript, which is a practical XSS advantage over storing session material in `localStorage`.
- Integrated chart/map libraries deliver FR-11 route visibility without building a custom GIS stack, reducing time-to-pilot for branch supervisors.

API Gateway

Item	Choice
Product	NGINX or Kong Gateway
Responsibilities	TLS termination, rate-limiting, IP allow-list, JWT validation, routing to Core Backend

Why these choices (advantages).

- A dedicated gateway is the **only** public network surface. Internal services (Core, Middleware, databases) stay on a private network — attackers and misconfigured clients cannot open PostgreSQL or Redis ports from the internet.
- **TLS termination at the edge** encrypts all field traffic once, instead of leaving each microservice to manage certificates inconsistently.
- Built-in **rate limiting** stops flood traffic before it reaches the JVM, which is more efficient than relying on application-level rejection alone under Denial-of-Service pressure.
- Path-based routing exposes only documented APIs; unknown paths never reach business logic.

Core Backend (Business Logic)

Item	Choice
Runtime	Java 21 (LTS)
Framework	Spring Boot 4
Reactive layer	Spring WebFlux (sync ingestion, notifications, auth filters)
Persistence	Spring Data JPA + Hibernate (transactions, savings, loans — ACID)
Security	Spring Security + JWT (jjwt)
Messaging client	Spring AMQP (RabbitMQ)
HTTP client to MW	Spring WebClient
Build	Maven or Gradle

Why these choices (advantages).

- **Spring Boot 4** ships an integrated stack (web, data, messaging, metrics) so transaction boundaries, security filters and outbound messaging are first-class framework concerns — not glued together from unrelated libraries.
- **Spring Security** provides battle-tested authentication/authorisation filters, method-level access control and JWT integration. For a system that moves money (escrow, collections, OFJ, activation fees), this is essential: roles (agent, client, cashier, admin) and branch scope can be enforced *before* business code runs.
- **Spring Data JPA + Hibernate** give declarative **ACID** transactions: escrow decrement, collection insert and audit row either all commit or all roll back — critical for financial consistency under concurrent sync.
- **WebFlux** sustains many concurrent mobile connections (offline flushes, notifications) with non-blocking I/O, which is advantageous when dozens of agents reconnect at once after a market blackout.
- **Spring AMQP** publishes CBS work asynchronously so a slow or down Core Banking link does not freeze the agent's HTTP request — a structural advantage over synchronous request/response to the bank on every collection.
- **Java 21 LTS** guarantees long support windows suitable for institutional software, with strong static typing that catches many integration errors at compile time.

CBS Middleware (Integration Service)

Item	Choice
Runtime	Java 21 (LTS)
Framework	Spring Boot 4
Persistence	Spring Data JPA (mapping logs, retry state, idempotency keys)
Messaging	Spring AMQP consumer (listens to RabbitMQ queues from Core)
Outbound CBS	RestTemplate / WebClient adapters per CBS vendor (Amplitude, FinanSoft, ...)
Retry / async	Spring Retry + optional Spring Batch for bulk daily exports

Why these choices (advantages).

- A **dedicated Spring Boot service** for CBS integration isolates Amplitude/FinanSoft (and other) protocols so a vendor change does not force a rewrite of Core business rules — the main advantage of a true adapter boundary.
- **Same stack as Core (Spring Boot 4 + JPA + Spring Security patterns)** means one security model, one metrics/tracing pipeline, and shared operational knowledge. Idempotency keys and outbound audit rows are written in the same transactional style as Core money movements.
- **Spring Retry** and queue redelivery absorb transient CBS outages: the field collection is already accepted; only the bank post is retried. That is safer for agents than failing the

whole operation when the CBS is slow.

- Per-vendor **WebClient/RestTemplate adapters** keep each Core Banking dialect in one module, which scales better than scattering `if (vendor == ...)` branches inside Core controllers.

Data & Messaging Infrastructure

Item	Choice
Primary database	PostgreSQL 16 (Core Backend schema)
Middleware DB	PostgreSQL 16 (separate schema or database for adapter state)
Cache / sessions	Redis 7
Message broker	RabbitMQ 3 (AMQP) — offline sync bursts, CBS job queue

Why these choices (advantages).

- **PostgreSQL** offers strong consistency, rich constraints and proven behaviour under concurrent writes — appropriate for ledgers where amounts are stored as integer XAF and financial facts are never soft-deleted. Open licensing avoids vendor lock-in for a research/pilot deployment.
- **Separate middleware schema/database** ensures CBS retry and mapping tables cannot corrupt or lock core collection rows — failure isolation that a single mixed schema would not provide.
- **Redis** answers session and hot-key reads in memory, shielding PostgreSQL from repetitive JWT validation and reference-data traffic during peak sync windows.
- **RabbitMQ (AMQP)** provides durable, acknowledged queues: when many agents leave a dead zone together, bursts land in the broker instead of exhausting Core threads or the CBS. Mobile clients get a fast 202-style path; heavy work proceeds asynchronously — a clear advantage over pure synchronous processing end-to-end.

Containerisation & Ops

Item	Choice
Containers	Docker + Docker Compose
Orchestration (prod)	Optional Kubernetes later; Compose is sufficient for pilot
Secrets	Environment variables / Docker secrets

Why these choices (advantages).

- **Docker Compose** runs the entire dependency graph (database, cache, broker, both Spring services, gateway) with one command, which is a practical advantage for a small academic/pilot team versus adopting a full orchestrator too early.

- Environment-injected **secrets** keep CBS API keys and JWT signing material out of the repository — mandatory for any integration that talks to a real Core Banking System.
- The same Compose file becomes the reference topology for demos and acceptance tests, reducing “works on my machine” drift.

Observability Stack

The platform uses the Grafana stack (Prometheus, Grafana, Loki, Tempo) for full-stack observability across Core Backend and CBS Middleware.

Component	Role in MICROFI
Prometheus	Metrics scrape target. Spring Boot Actuator + Micrometer expose /actuator/prometheus on Core and Middleware (JVM, HTTP, RabbitMQ, JPA, custom business counters such as collections/min, OFJ variance, token activations).
Grafana	Single pane of glass: dashboards for API latency, error rates, escrow locks, sync backlog, CBS call success rate. Alerting rules (e.g. SOS spike, CBS downtime).
Loki	Log aggregation. Structured JSON logs from both Spring Boot services (correlation ID, agent ID, branch ID) via Promtail or Grafana Alloy.
Tempo	Distributed tracing. OpenTelemetry instrumentation on Core → RabbitMQ → Middleware → CBS so a single collection can be traced end-to-end.
Instrumentation	Micrometer (metrics) + OpenTelemetry Java agent / Spring Observability (traces) + Logback JSON encoder (logs). Shared traceId / correlationId across all three signals.

Why this stack (advantages).

- **Grafana + Prometheus + Loki + Tempo** unifies metrics, logs and traces in one console. Operators diagnose a failed CBS post or a stuck offline sync without switching between disconnected tools.
- **Micrometer / Actuator** on Spring Boot expose both JVM health and *business* counters (collections/min, escrow lockouts, OFJ variance, sync lag) as first-class Prometheus series — stronger for pilot KPIs than infrastructure-only monitoring.
- **Tempo + OpenTelemetry** reconstructs a single collection path Core → queue → Middleware → CBS, which is decisive when debugging intermittent bank integration failures.
- The stack is lightweight enough for a pilot yet standard enough to grow if the deployment later moves to a larger ops footprint.

4. Component Breakdown Summary

Layer	Responsibilities	Technology
Client	Mobile dual views (Agent + Client); offline encrypted store; Back-Office dashboards.	Flutter 3 / Next.js 16
Gateway	TLS, rate-limit, JWT, routing.	NGINX or Kong
Core Backend	Business rules, offline sync, escrow, OFJ, tokens, notifications.	Spring Boot 4 WebFlux + JPA
CBS Middleware	Vendor-specific CBS mapping, posting, fee split, balance fetch, retries.	Spring Boot 4 + JPA
Data / Cache	Relational state + session/token cache.	PostgreSQL 16 + Redis 7
Async	Decouple Core ↔ Middleware; offline burst processing.	RabbitMQ

Table 1: MICROFI Component Breakdown

5. Internal Modular Structure (Core Backend)

The Core Backend is a modular monolith. Modules communicate only through public service interfaces.

Hybrid model: **WebFlux** for sync, notifications, authentication and outbound calls toward the Middleware. **Spring Data JPA** for the financial ledger, savings accounts and loan entities (strict ACID).

Directory Layout

```
microfi-core/
|-- src/main/java/com/microfi/
|   |-- infrastructure/      # RabbitMQ, Redis, DataSource
|   |-- shared/              # DTOs, exceptions, utilities
|
|   # --- REACTIVE (WebFlux) ---
|   |-- synchronization/    # Offline sync ingestion
|   |-- notifications/      # SMS / push
|   |-- authentication/     # JWT, IMEI binding
|   |-- cbsclient/          # WebClient -> Middleware
|
|   # --- BLOCKING (Data JPA / ACID) ---
|   |-- transactions/       # Ledger, denominations
|   |-- savings/            # Balances, digital booklet tokens
```

```
|   +-- loans/                               # Schedules & disbursements
```

Middleware Service Layout

```
microfi-middleware/  
|-- src/main/java/com/microfi/mw/  
|   |-- infrastructure/      # RabbitMQ listeners, DataSource  
|   |-- adapters/           # One package per CBS vendor  
|   |   |-- amplitude/  
|   |   |-- finansoft/  
|   |   +-- generic/  
|   |-- domain/             # Idempotency keys, mapping logs (JPA)  
|   +-- api/                # Internal REST exposed to Core only
```

Core Module Responsibilities

Module	Responsibility	Engine
synchronization	Offline batch ingest, conflict resolution	WebFlux
notifications	Flash SMS, push, receipt triggers	WebFlux
authentication	JWT, IMEI binding, session lifecycle	WebFlux
cbsclient	Publish jobs to RabbitMQ / call Middleware REST	WebFlux
transactions	Atomic ledger, denomination, variance	JPA
savings	Balances, activation tokens, fee split orchestration	JPA
loans	Loan schedules & disbursements	JPA

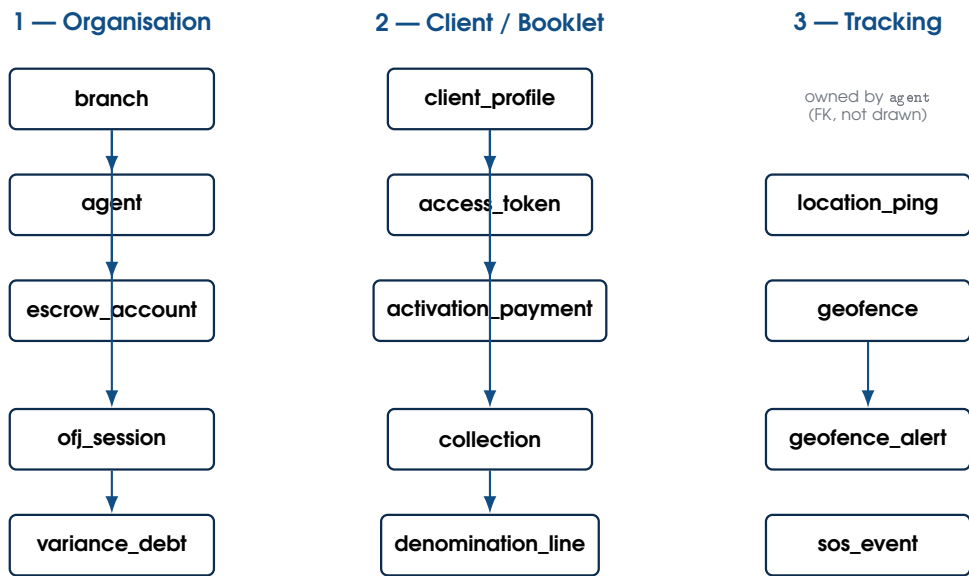
6. Database Schema

The Core Backend and the CBS Middleware share the same PostgreSQL instance but use **separate schemas** (*core* and *mw*) to preserve clear ownership boundaries. All monetary amounts are stored as integers in the smallest currency unit (XAF). Timestamps are stored in UTC (TIMESTAMPTZ).

Design rules: (1) No soft-deletes on financial facts — use status enums and immutable audit rows. (2) Denomination breakdown is stored as structured JSONB *and* normalised lines for reporting. (3) Client digital tokens never lock the underlying CBS balance; expiry only restricts mobile transactional APIs.

Entity Relationship Overview

Entities are grouped into three independent domain columns. Relationships are strictly **top** → **bottom** inside each column. Cross-domain foreign keys (*collection.agent_id*, *activation_payment.agent_id*) are noted in the table list below and are *not* drawn as crossing edges.



Cross-domain FKs (not drawn): `collection.agent_id → agent`; `activation_payment.agent_id → agent`; `location_ping.agent_id, sos_event.agent_id, geofence.agent_id → agent`; `ofj_agent_line.agent_id → agent`. Keeping these as documentation-only avoids the spaghetti that appears when every FK is a visible edge.

Schema core — Identity & Organisation

Table	Key columns	Notes
branch	id, code, name, open_time, close_time, timezone	FR-15 schedule windows
agent	id, branch_id, employee_code, full_name, phone, imei, status, pin_hash	FR-01 IMEI binding; status ∈ {ACTIVE, SUSPENDED}
admin_user	id, login, role, branch_id_nullable	Back-Office RBAC

Schema core — Escrow (FR-03, FR-04, FR-05)

Table	Key columns	Notes
escrow_account	id, agent_id, balance_xaf, ceiling_xaf, updated_at	Running guarantee; lockout when cumulative cash ≥ ceiling
escrow_ledger	id, escrow_id, delta_xaf, reason, ref, created_at	Top-ups, adjustments; immutable
ceiling_override	id, agent_id, temp_ceiling_xaf, reason, valid_until, approved_by	FR-05 temporary override

Schema core — Client Digital Booklet (FR-19–FR-22)

Table	Key columns	Notes
client_profile	id, mfi_member_no, full_name, phone, branch_id, cbs_ref, status	Local mirror of CBS member; no CBS create
activation_id	id, client_id, code_hash, expires_at, used_at	Single-use temporary ID from CBS (FR-19)
access_token	id, client_id, issued_at, expires_at, status	365-day digital booklet token
activation_payment	id, client_id, agent_id, amount_xaf, agent_commission_xaf, mfi_share_xaf, paid_at	Fee split audit (FR-19)

Schema core — Collections & Tracking (FR-06–FR-14)

Table	Key columns	Notes
collection	id, agent_id, client_id, amount_xaf, lat, lon, accuracy_m, collected_at, sync_status, device_tx_id	Offline-safe; unique (agent_id, device_tx_id); lat/lon mandatory (BR-05)
denomination_line	id, collection_id, face_value_xaf, quantity	FR-08: 500, 1k, 2k, 5k, 10k + coins aggregate
location_ping	id, agent_id, lat, lon, recorded_at	FR-10 periodic GPS
geofence	id, agent_id or branch_id, polygon geography	FR-13 assigned perimeter
geofence_alert	id, agent_id, geofence_id, raised_at, resolved_at	Active breach list
sos_event	id, agent_id, lat, lon, raised_at, acknowledged_by	FR-14 distress
notification_log	id, collection_id, channel, status, sent_at	FR-09 SMS / print flag

Schema core — End-of-Day OFJ (FR-16–FR-18)

Table	Key columns	Notes
ofj_session	id, branch_id, business_date, status, opened_at, closed_at	One open session per branch/day
ofj_agent_line	id, ofj_id, agent_id, digital_total_xaf, physical_total_xaf, delta_xaf	Δ = physical – digital (BR-01)
ofj_physical_denom	id, ofj_agent_line_id, face_value_xaf, quantity	Cashier-counted breakdown
variance_debt	id, agent_id, ofj_agent_line_id, amount_xaf, status, created_at	FR-17 agent debt
export_batch	id, ofj_id, file_uri, format, generated_at, ack_status	FR-18 CBS export

Schema _{mw} — Middleware Idempotency & Mapping

Table	Key columns	Notes
idempotency_key	key, operation, request_hash, response_body, created_at	Prevents double-post to CBS
cbs_call_log	id, correlation_id, operation, vendor, http_status, duration_ms, created_at	Full audit of outbound CBS calls
outbox_job	id, type, payload jsonb, status, attempts, next_retry_at	RabbitMQ redelivery companion

Illustrative DDL (excerpts)

```
-- core.collection (offline-safe)
CREATE TABLE core.collection (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id    UUID NOT NULL REFERENCES core.agent(id),
  client_id   UUID NOT NULL REFERENCES core.client_profile(id),
  amount_xaf  BIGINT NOT NULL CHECK (amount_xaf > 0),
  lat         DOUBLE PRECISION NOT NULL, -- BR-05: GPS mandatory
  lon         DOUBLE PRECISION NOT NULL,
  accuracy_m  REAL,
  collected_at TIMESTAMPTZ NOT NULL,
  sync_status TEXT NOT NULL DEFAULT 'PENDING',
  device_tx_id TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (agent_id, device_tx_id)
);

-- core.access_token (digital booklet)
CREATE TABLE core.access_token (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id   UUID NOT NULL REFERENCES core.client_profile(id),
  issued_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at  TIMESTAMPTZ NOT NULL,
  status      TEXT NOT NULL DEFAULT 'ACTIVE'
  CHECK (status IN ('ACTIVE', 'GRACE', 'EXPIRED', 'REVOKED'))
);

-- mw.idempotency_key
CREATE TABLE mw.idempotency_key (
  key          TEXT PRIMARY KEY,
  operation     TEXT NOT NULL,
  request_hash  TEXT NOT NULL,
  response_body JSONB,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
```


Indexing guidance: btree on collection(agent_id, collected_at), collection(client_id, collected_at), location_ping(agent_id, recorded_at), access_token(client_id, status), and partial unique index on ofj_session(branch_id, business_date) where status <> CANCELLED.

7. STRIDE Security Matrix

This section maps the classical STRIDE threat model to MICROFI's architecture (Flutter edge, API Gateway, Spring Boot 4 Core + Middleware, RabbitMQ, PostgreSQL, CBS). Each threat category is linked to the security property it violates, the concrete system risks, and the design controls already specified in the technology stack, schema and API surface.

Scope: Agent terminals (IMEI-bound), Client app (JWT + digital booklet token), Back-Office, Core Backend, CBS Middleware, message broker and data at rest. Residual risk remains where the platform depends on the external CBS and on physical custody of cash.

Summary Matrix

STRIDE	Property	Focus	MICROFI risk mapping
Spoofing	Authenticity	Impersonating a user or component	Device spoofing if IMEI binding fails; JWT/token theft impersonating agent, client or admin.
Tampering	Integrity	Unauthorized modification of data or code	Tampering with offline AES-encrypted payloads; alteration of ledger facts or audit logs.
Repudiation	Non-repudiability	Denying an action without proof	Agent denying a collection or variance debt if geotag (FR-12) or sync logs are incomplete; missing audit on ceiling overrides (FR-05).
Information Disclosure	Confidentiality	Leaking sensitive data	PII, balances or booklet tokens exposed in transit (AMQP/API); insecure CBS credentials in adapters.
Denial of Service	Availability	Making a resource unavailable	Flooding gateway or offline sync queues; CBS outage causing sync backlog (buffered by RabbitMQ).
Elevation of Privilege	Authorization	Gaining higher privileges	BOLA across branches; race conditions during offline sync bursts to bypass escrow ceiling (BR-03).

S — Spoofing (Authenticity)

Item	Detail
Threats	Unauthorized use of an agent terminal if IMEI binding is circumvented; stolen or forged JWT presenting as a valid agent, client or back-office user; reuse of a consumed Activation ID.
Design controls	FR-01 IMEI-bound SSO; Spring Security + JWT with short TTL and refresh; biometric/PIN on device; single-use hashed Activation IDs; Gateway JWT validation before routing.
Residual risk	Rooted devices, GPS spoofing apps, or physical SIM/terminal cloning outside pure software control.

T — Tampering (Integrity)

Item	Detail
Threats	Modification of local offline collection payloads or SQLCipher/Hive stores on a compromised handset; unauthorized change of historical ledger rows or audit trails.
Design controls	AES-256 encrypted offline queue (NFR-01); immutable financial facts (status + append-only audit, no soft-delete of money); device_tx_id uniqueness; denomination checksum; middleware idempotency keys before CBS post.
Residual risk	Advanced malware with key access on a jailbroken device before sync.

R — Repudiation (Non-repudiability)

Item	Detail
Threats	Agent denies a cash collection or variance debt; denial of a temporary ceiling override or schedule change if trails are incomplete.
Design controls	BR-05 mandatory GPS gate; FR-12 invisible geotag on every validated deposit; OFJ Δ and variance_debt tables; mandatory justification + approver on ceiling overrides (FR-05); correlation IDs end-to-end; Loki log retention with agent/branch identifiers.
Residual risk	BR-05 / FR-12 hard-block collection when GPS is denied; residual risk is limited to agents who cannot work until location is enabled (operational, not silent unlocated cash).

I — Information Disclosure (Confidentiality)

Item	Detail
Threats	Leak of PII, balances or digital booklet tokens in transit; exposure of CBS API credentials in middleware configuration.
Design controls	TLS at the Gateway; TLS or mTLS for service-to-service and AMQP; secrets via environment / Docker secrets (never hardcoded); branch-scoped data access in Back-Office; token expiry does not expose extra fields beyond read-only balance/history.
Residual risk	Misconfigured RabbitMQ management UI or Grafana left internet-exposed without auth.

D — Denial of Service (Availability)

Item	Detail
Threats	Flooding public API endpoints or offline sync ingress; CBS slowdown or outage creating cascading backlog.
Design controls	Gateway rate-limiting and IP allow-list; RabbitMQ buffering between Core and Middleware; offline-first agent app continues local capture; Spring Retry + outbox for CBS; Prometheus/Grafana alerts on queue depth and error rate.
Residual risk	Prolonged CBS outage delays final ledger posting though field operations continue.

E — Elevation of Privilege (Authorization)

Item	Detail
Threats	Broken object-level authorization (BOLA) allowing access to another branch's agents, clients or OFJ; concurrent offline sync races that exceed escrow ceiling (BR-03).
Design controls	Server-side branch scope on every admin query; RBAC for admin roles; escrow lock evaluated on server at sync commit (not only on device); optimistic locking / transactional check of cumulative cash vs ceiling; agent status SUSPENDED invalidates sessions.
Residual risk	Insider threat with legitimate elevated Back-Office credentials (process control, not software alone).

Traceability to Design Artifacts

STRIDE theme	Primary design anchors
Spoofing	FR-01, authentication module, Gateway JWT, Activation ID lifecycle
Tampering	NFR-01, <code>core.collection</code> + denomination lines, <code>mw.idempotency_key</code>
Repudiation	FR-05, FR-12, FR-16/17, Loki correlation, <code>cbs_call_log</code>
Disclosure	TLS, Docker secrets, schema separation <code>core/mw</code>
Denial of Service	Rate-limit, RabbitMQ, observability (Prometheus/Grafana/Loki/Tempo)
Elevation	Branch-scoped APIs, BR-03 server-side escrow, RBAC

8. Use Case Descriptions (from Business Analysis)

The following use-case texts are carried forward from the Business Analysis & Requirements Specification so that architecture, APIs and schema remain traceable to behavioural specifications. Format: **ID, Name, Actors, Objectives, Preconditions, Postconditions, Nominal Scenario, Alternative Scenarios, Business Rules.**

Agent Access & Administration

UC-01 — Agent Authentication via IMEI-bound SSO

Actors: Collection Agent, System.

Objectives: Authenticate the agent and bind the session to the enrolled terminal.

Preconditions: Agent enrolled; IMEI registered; PIN/biometric available.

Postconditions: Secure session open; audit log written.

Nominal: Launch app → read IMEI → enter credentials → validate IMEI + profile → dashboard.

Alternatives: Unknown IMEI (reject); invalid credentials (lock after 3 tries); offline auth against encrypted cache.

Rules: Session limited to schedule window (FR-15); IMEI binding mandatory (FR-01).

UC-02 — Agent Profile Enrollment & Lifecycle

Actors: System Administrator, Branch Manager.

Objectives: Maintain registry of agents and terminals.

Preconditions: Admin authenticated; branch exists.

Postconditions: Profile created/updated/suspended; IMEI linked; audited.

Nominal: Open Agent Management → enter identity, branch, escrow ref, IMEI → validate uniqueness → activate.

Alternatives: Duplicate IMEI; suspend (invalidate sessions); soft-delete only.

Rules: Privilege required; no delete with open sessions or unresolved variance (FR-02).

UC-15 — Branch Working-Hours Configuration

Actors: Branch Manager / Administrator.

Objectives: Enforce temporal boundaries of collection and GPS privacy.

Preconditions: Schedule privilege.

Postconditions: Windows stored; app refuses collections outside window; tracking off outside hours (NFR-10).

Nominal: Configure open/close/grace → push to agents → enforce on terminal.

Alternatives: Late collection blocked; emergency override by privileged role.

Rules: FR-15; schedule history retained.

Escrow & Ceilings

UC-03 — Escrow Wallet Top-Up

Actors: Agent, Administrator, payment channel / CBS.

Objectives: Increase escrow balance and ceiling.

Preconditions: Active agent; channel configured.

Postconditions: Balance increased; ledger entry; visible on Back-Office.

Nominal: Initiate top-up → select channel → confirm payment → credit escrow.

Alternatives: Payment failure; offline queue; manual cashier top-up (MVP).

Rules: FR-03; positive amounts only.

UC-04 — Automatic Collection Lockout

Actors: System, Collection Agent.

Objectives: Block collection beyond financial guarantee.

Preconditions: Authenticated; active escrow.

Postconditions: New collections blocked if cumulative cash $\geq$ escrow; agent notified.

Nominal: Start collection → compare cumulative vs remaining → allow or lock.

Alternatives: Partial residual; concurrent top-up releases lock.

Rules: BR-03; offline uses last synced ceiling (FR-04).

UC-05 — Remote Temporary Ceiling Override

Actors: Branch Manager / Administrator.

Objectives: Temporary audited increase of ceiling.

Preconditions: Privilege; justification.

Postconditions: Temp ceiling applied; auto-restore; audit.

Nominal: Open live status → adjust ceiling + reason → push to terminal.

Alternatives: Insufficient privilege; expiry restores original.

Rules: FR-05; mandatory justification.

Collection Operations

UC-06 — Multi-Channel Client Lookup

Actors: Collection Agent, System.

Objectives: Resolve a unique client before any deposit.

Preconditions: Authenticated; registry online or cached.

Postconditions: Single client locked for the transaction.

Nominal: Choose QR / ID / phone / name → query → confirm unique profile.

Alternatives: Multiple matches; no match; offline partial cache.

Rules: FR-06; no validation without unique client.

UC-07 — Secure Offline Collection Recording

Actors: Collection Agent, System (local store).

Objectives: Persist collections without network.

Preconditions: Authenticated; encrypted local store ready; **GPS active with valid fix (BR-05).**

Postconditions: Encrypted pending record; local totals updated.

Nominal: Collect offline → write encrypted payload (incl. GPS) → confirm saved.

Alternatives: Storage full; GPS off → block collection; connectivity restored → background sync.

Rules: FR-07; BR-05; NFR-01 AES-256; NFR-08 non-blocking sync.

UC-08 — Mandatory Denomination Breakdown

Actors: Collection Agent, System.

Objectives: Capture physical note composition.

Preconditions: Client identified; amount above threshold.

Postconditions: Breakdown stored; sum equals amount.

Nominal: Enter quantities per face value → live total → confirm equality.

Alternatives: Sum mismatch blocks; below threshold optional.

Rules: BR-02; FR-08.

UC-09 — Post-Validation Multi-Channel Notification

Actors: System, SMS gateway, printer, Client, Agent.

Objectives: Instant proof of deposit.

Preconditions: Transaction validated.

Postconditions: SMS queued/sent; receipt printed if available.

Nominal: Validate → Flash SMS → optional Bluetooth print.

Alternatives: SMS failure (retry); no printer; offline queue.

Rules: FR-09; NFR-05 latency target online.

UC-12 — Mandatory GPS Gate & Transaction Geotagging

Actors: System, Collection Agent.

Objectives: Block collection without GPS; stamp every allowed deposit with immutable location/time.

Preconditions: Agent starts collection; GPS must be ON with a valid fix (BR-05).

Postconditions: Collection blocked, or lat/lon/accuracy/timestamp stored.

Nominal: Verify GPS active → capture fix on validate → attach to payload.

Alternatives: GPS off / no fix → **hard block** (no collection); coarse fix allowed and marked.

Rules: BR-05; FR-12; geotag non-editable by agent.

Field Tracking & Security

UC-10 — Periodic GPS Position Reporting

Actors: Mobile system, Back-Office, Branch Manager.

Objectives: Live trail of active agents.

Preconditions: Session open; location allowed; within service hours.

Postconditions: Points stored and shown on map.

Nominal: Sample every ~5 min → send or queue.

Alternatives: Location disabled; outside hours (NFR-10 stop).

Rules: FR-10; NFR-09 battery optimisation.

UC-11 — Historical Route Visualization

Actors: Branch Manager / Administrator.

Objectives: Verify market coverage.

Preconditions: Pings recorded for agent/date.

Postconditions: Map with polyline and transaction markers.

Nominal: Select agent/date → render route.

Alternatives: Sparse data; no data.

Rules: FR-11; branch-scoped access.

UC-13 — Real-Time Geofence Breach Alert

Actors: System, Branch Manager.

Objectives: Detect out-of-zone activity.

Preconditions: Assigned geofence; tracking active.

Postconditions: Console alert; logged.
Nominal: Evaluate position vs polygon → raise alert.
Alternatives: Grace period; auto-clear on return.
Rules: FR-13.

UC-14 — Emergency SOS Activation

Actors: Collection Agent, Branch Manager, System.
Objectives: High-priority location alert.
Preconditions: Authenticated; SOS contacts configured.
Postconditions: SMS + Back-Office critical event.
Nominal: Discrete SOS → capture GPS → notify.
Alternatives: No network (SMS still attempted); false positive handling.
Rules: FR-14; cannot be disabled by agent.

End-of-Day (OFJ)

UC-16 — Digital Cash Desk / OFJ Reconciliation

Actors: Branch Cashier, Agent, System.
Objectives: Compute Δ and close the day.
Preconditions: Offline txs synced; cashier authenticated.
Postconditions: Δ displayed; session closed if $\Delta = 0$ or after regularisation.
Nominal: Open OFJ → enter physical denominations → $\Delta = \text{physical} - \text{digital}$ → validate.
Alternatives: Negative Δ blocks close (ticket); positive Δ surplus note.
Rules: BR-01; FR-16; no close with pending sync.

UC-17 — Cash Variance as Agent Debt

Actors: Branch Manager / Administrator.
Objectives: Formalise shortage against agent.
Preconditions: Non-zero variance; privilege.
Postconditions: Debt on profile; OFJ may close.
Nominal: Record as debt → comment → confirm.
Alternatives: Partial recovery; write-off workflow.
Rules: FR-17; shortages only; immutable audit.

UC-18 — Daily Accounting Export

Actors: Cashier / Admin, System, CBS.
Objectives: CBS-compatible daily file.
Preconditions: OFJ closed; format configured.
Postconditions: File generated; logged.
Nominal: Trigger export → map transactions → store/push.
Alternatives: Mapping error; interim export if policy allows.
Rules: FR-18; closed sessions only.

Client Digital Booklet & Self-Service

UC-19 — Tokenized Client Activation / Annual Renewal

Actors: Existing Client, Field Agent, System, CBS.

Objectives: Activate/renew digital profile; fee split; 365-day token.

Preconditions: Temporary Activation ID from CBS; agent authenticated.

Postconditions: Profile Active; fee split in CBS; Activation ID consumed.

Nominal (first): Enter Activation ID → set credentials → Inactive → agent sponsors → pay fee → split → issue token.

Nominal (renewal): Token expired (read-only) → agent sponsors renewal → new 365-day token.

Alternatives: ID not found (redirect to physical registration, no CBS create); already active; grace period; payment failure; invalid agent.

Rules: BR-04; money never locked; optional debit from savings with PIN.

UC-20 — Client Profile Self-Service View

Actors: Client, System.

Objectives: Show identity and booklet status.

Preconditions: Activated or read-only after expiry.

Postconditions: Profile displayed; no modification.

Nominal: Open My Profile → show name, membership, phone, branch, token status/expiry.

Alternatives: Offline snapshot; expired token still visible.

Rules: FR-20; client cannot edit identity attributes.

UC-21 — Account Balance & Summary

Actors: Client, System, CBS.

Objectives: Continuous visibility of savings.

Preconditions: Authenticated or read-only mode.

Postconditions: Balance/summary shown; no debit/credit.

Nominal: Open My Account → cache/CBS balance → YTD summary.

Alternatives: Offline last-known; token expired still shows balance; CBS down → cache.

Rules: FR-21; balance never blocked by token expiry.

UC-22 — Contribution History & Mini-Statement

Actors: Client, System.

Objectives: Digital proof replacing paper booklet pages.

Preconditions: Authenticated or read-only; history exists.

Postconditions: History or mini-statement shown/sent.

Nominal (app): History list with filters.

Nominal (USSD): Short code → last N contributions.

Alternatives: Empty state; offline cache; expired token still consultable.

Rules: FR-22; append-only from client perspective.

Traceability: UC-01..UC-22 map to FR-01..FR-22 and drive the API catalogue (Section on Key API Endpoints) and the core/mw schema. Implementation priority follows the Feature

Priority Matrix (MVP vs Fast-Follow).

9. Human-Machine Interface (HMI) Design Choices

FIELD AGENT MOBILE INTERFACE

“Fat-Finger” & Low-Literacy Optimization. High-contrast colours, large touch targets (min. 48 dp), 3-step collection wizard, persistent sync-status banner, Bluetooth thermal printer. **GPS gate:** if location services are off or no valid fix is available, the collection wizard does not open and the agent is prompted to enable GPS (BR-05 / FR-12).

CLIENT MOBILE INTERFACE

Consumer-grade banking UI: balance, digital booklet status, contribution history, profile. **Read-only mode** after token expiry — money visibility is never locked.

BACK OFFICE WEB INTERFACE

High-density grids, liquidity charts, confirmation modals for high-risk actions, geolocation maps, audit logs.

10. Production-Ready Docker Compose Configuration

```
version: '3.8'

services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: microfi_db
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - microfi-net

  redis:
    image: redis:7-alpine
    networks:
      - microfi-net

  rabbitmq:
    image: rabbitmq:3-management-alpine
    ports:
      - "15672:15672"
    networks:
      - microfi-net

  core-backend:
    build: ./microfi-core
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/microfi_db
      - SPRING_DATASOURCE_USERNAME=${DB_USER}
      - SPRING_DATASOURCE_PASSWORD=${DB_PASSWORD}
      - REDIS_HOST=redis
      - RABBITMQ_HOST=rabbitmq
      - MIDDLEWARE_BASE_URL=http://middleware:8081
    depends_on:
      - postgres
      - redis
      - rabbitmq
    networks:
      - microfi-net

  middleware:
    build: ./microfi-middleware
    environment:
      - SPRING_PROFILES_ACTIVE=docker
      - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/microfi_db
```

```
- SPRING_DATASOURCE_USERNAME=${DB_USER}
- SPRING_DATASOURCE_PASSWORD=${DB_PASSWORD}
- RABBITMQ_HOST=rabbitmq
- CBS_ENDPOINT=${CBS_API_URL}
- CBS_VENDOR=${CBS_VENDOR}
depends_on:
  - postgres
  - rabbitmq
networks:
  - microfi-net

backoffice:
  build: ./backoffice
  ports:
    - "3000:3000"
  networks:
    - microfi-net

# --- Observability: Grafana + Prometheus + Loki + Tempo ---
prometheus:
  image: prom/prometheus:v2.54.1
  volumes:
    - ./observability/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - promdata:/prometheus
  ports:
    - "9090:9090"
  networks:
    - microfi-net

loki:
  image: grafana/loki:3.1.1
  command: -config.file=/etc/loki/local-config.yaml
  ports:
    - "3100:3100"
  networks:
    - microfi-net

tempo:
  image: grafana/tempo:2.6.1
  command: [ "-config.file=/etc/tempo.yaml" ]
  volumes:
    - ./observability/tempo.yaml:/etc/tempo.yaml:ro
  ports:
    - "3200:3200"    # Tempo API
    - "4317:4317"    # OTLP gRPC
    - "4318:4318"    # OTLP HTTP
  networks:
    - microfi-net

grafana:
  image: grafana/grafana:11.2.0
```

```
environment:
  - GF_SECURITY_ADMIN_USER=${GRAFANA_USER:-admin}
  - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD:-admin}
volumes:
  - grafanadata:/var/lib/grafana
  - ./observability/grafana/provisioning:/etc/grafana/provisioning:ro
ports:
  - "3001:3000"
depends_on:
  - prometheus
  - loki
  - tempo
networks:
  - microfi-net

alloy: # optional: scrapes logs + forwards OTLP
  image: grafana/alloy:v1.4.2
  volumes:
    - ./observability/alloy.config:/etc/alloy/config.alloy:ro
    - /var/log:/var/log:ro
  depends_on:
    - loki
    - tempo
  networks:
    - microfi-net

volumes:
  pgdata:
  promdata:
  grafanadata:

networks:
  microfi-net:
    driver: bridge
```

Both core-backend and middleware are Spring Boot 4 services exposing /actuator/prometheus and OTLP traces. Credentials and CBS_API_URL / CBS_VENDOR are injected via environment variables. Grafana is published on port 3001 to avoid clashing with the Back Office on 3000. The Flutter app is distributed separately.

11. Key API Endpoints

Endpoints follow FR-01 to FR-22. Two surfaces:

- **Core Backend** (/api/v1/...) — consumed by Agent app, Client app, Back-Office.
- **CBS Middleware** (/mw/v1/...) — internal only; called by Core via WebClient or RabbitMQ jobs.

Core Backend (API Gateway surface)

Authentication & Agents (FR-01, FR-02, FR-15)

Endpoint	Purpose	FR
POST /auth/agent/login	Agent SSO (IMEI + PIN/biometric)	FR-01
POST /auth/client/activate	First login with temporary Activation ID	FR-19
POST /auth/client/login	Client login after activation	FR-19
GET/POST /admin/agents	Agent enrollment CRUD	FR-02
PATCH /admin/agents/{id}/status	Suspend / reactivate	FR-02
GET/PUT /admin/branches/{id}/schedule	Session time windows	FR-15

Escrow & Collection (FR-03–FR-09, FR-12)

Endpoint	Purpose	FR
GET /agents/{id}/escrow	Escrow balance & ceiling	FR-04
POST /agents/{id}/escrow/top-up	Top-up record / callback	FR-03
PATCH /admin/agents/{id}/ceiling	Remote ceiling override	FR-05
GET /clients/lookup	Multi-method client identification	FR-06
POST /collections	Deposit + denomination + geotag; rejected if GPS missing	FR-07,08,12
POST /collections/sync	Offline batch upload	FR-07
POST /collections/{id}/notify	SMS / receipt flag	FR-09

Tracking, OFJ, Client Self-Service (FR-10–FR-22)

Endpoint	Purpose	FR
POST /agents/{id}/location	GPS ping	FR-10
GET /admin/agents/{id}/route	Daily route	FR-11
POST /agents/{id}/sos	Distress alert	FR-14
GET /ofj/{branchId}/summary	Digital totals for reconciliation	FR-16
POST /ofj/{branchId}/reconcile	Physical counts → variance Δ	FR-16
POST /ofj/{branchId}/variance	Agent debt	FR-17
POST /ofj/{branchId}/export	Daily CBS export file	FR-18
POST /clients/activation	Token activation / renewal + fee split	FR-19
GET /clients/me/profile	Profile consultation	FR-20
GET /clients/me/balance	Balance (visible if token expired)	FR-21
GET /clients/me/history	Contribution history	FR-22

CBS Middleware (Spring Boot 4 + JPA) — internal

Internal contract only. Core Backend publishes jobs to RabbitMQ or calls these REST endpoints. Middleware persists idempotency keys and mapping logs via JPA, then calls the concrete CBS.

Endpoint	Purpose	FR
POST /mw/v1/members/verify	Validate Activation ID / member in CBS	FR-19
POST /mw/v1/members/balance	Live balance from CBS	FR-21
POST /mw/v1/transactions/post	Post collection(s) to CBS ledger	FR-18
POST /mw/v1/fees/split	Split activation fee (agent + MFI)	FR-19
POST /mw/v1/escrow/credit	Credit agent escrow in CBS	FR-03
GET /mw/v1/members/{id}/history	History pull when cache stale	FR-22
POST /mw/v1/exports/daily	Daily export acknowledgement	FR-18

Traceability: Every middleware invocation carries a correlation ID from Core. Spring Retry (and queue redelivery via RabbitMQ) handles CBS outages without blocking the mobile path. JPA stores idempotency keys so duplicate deliveries never double-post to the CBS.